

MongoDB Users and Roles Assignment

Database: **insta_clone** | Users: **storyAdmin**, **feedViewer** | Tool: mongosh

Name: _____ Roll No.: _____ Date: _____

Note: The outputs below are the expected results. Steps 1-3 and 5 should be done while connected as an admin user, and the MongoDB server must be started with authentication enabled (- - auth or security.authorization: enabled), otherwise the permission error in Question 4 will not appear. Use your own passwords in place of the sample ones.

Question 1: Connect to MongoDB and create the insta_clone database

Connect to the MongoDB server using the mongo shell and create a new database called insta_clone.

Commands

```
mongosh "mongodb://localhost:27017"

use insta_clone
db.createCollection("posts")
db.createCollection("comments")
db.createCollection("stories")

show dbs
```

Output

```
switched to db insta_clone
{ ok: 1 } (shown once for each createCollection)
admin      40.00 KiB
config     72.00 KiB
insta_clone 12.00 KiB      <-- new database (size may differ)
local      72.00 KiB
```

Explanation: use insta_clone only switches to the database. MongoDB creates it for real when the first collection or document is stored, which is why the collections are created straight away. (If you are on an older setup, the command mongo works in place of mongosh.)

Question 2: Create the 'storyAdmin' user with the dbAdmin role

Inside insta_clone, use createUser to add a user named 'storyAdmin' with the 'dbAdmin' role for managing Instagram Stories data.

Commands

```
use insta_clone
db.createUser({
  user: "storyAdmin",
  pwd: "StoryAdmin@123",
  roles: [ { role: "dbAdmin", db: "insta_clone" } ]
})
```

Output

```
{ ok: 1 }
```

Explanation: The user is stored in the insta_clone database and can administer that database (for example, create indexes, view collection statistics and manage collections). Note that dbAdmin on its own does not allow reading or writing the documents in the collections.

Question 3: Create the read-only user 'feedViewer'

Create a read-only user called 'feedViewer' in `insta_clone` who can only view (not modify) the posts and comments collections.

Commands

```
use insta_clone
db.createUser({
  user: "feedViewer",
  pwd: "FeedViewer@123",
  roles: [ { role: "read", db: "insta_clone" } ]
})
```

Output

```
{ ok: 1 }
```

Explanation: The built-in read role allows only read actions such as `find` and `listCollections`. It gives no permission to insert, update or delete.

Optional (stricter): The read role applies to every collection in `insta_clone`, not just posts and comments. If the user must see only those two collections, create a custom role instead:

```
db.createRole({
  role: "postsCommentsReader",
  privileges: [
    { resource: { db: "insta_clone", collection: "posts" }, actions: ["find"] },
    { resource: { db: "insta_clone", collection: "comments" }, actions: ["find"] }
  ],
  roles: []
})
db.grantRolesToUser("feedViewer", [ "postsCommentsReader" ])
db.revokeRolesFromUser("feedViewer", [ { role: "read", db: "insta_clone" } ])
```

Question 4: Log in as 'feedViewer' and try to insert a document

Simulate authentication: log in as 'feedViewer' and try to insert a new document into the posts collection. Note the error message and explain why it occurs.

Commands

```
mongosh -u feedViewer -p --authenticationDatabase insta_clone insta_clone
(enter the password when prompted)

// Reading works
db.posts.find()

// Writing fails
db.posts.insertOne({ user: "riya", caption: "Sunset at Sabarmati", likes: 0 })
```

Error message

```
MongoServerError: not authorized on insta_clone to execute command
{ insert: "posts", documents: [ { user: "riya", ... } ], ordered: true, $db: "insta_clone" }
code: 13, codeName: 'Unauthorized'
```

Why it occurs: MongoDB uses role-based access control. The 'feedViewer' user has only the read role, which does not include the `insert` action. When the user tries to insert into posts, the server checks the user's privileges, finds no permission for that action, and rejects the command with error code 13 (Unauthorized). The exact wording of the message can differ slightly between shell versions.

Question 5: List users with db.getUsers() and compare the roles

Use `db.getUsers()` to list all users in `insta_clone` and describe the roles assigned to each. Then explain in your own words the difference between 'read' and 'dbAdmin'.

Commands

```
use insta_clone
db.getUsers()
```

Output (shortened)

```
{
  users: [
    {
      _id: 'insta_clone.storyAdmin',
      user: 'storyAdmin',
      db: 'insta_clone',
      roles: [ { role: 'dbAdmin', db: 'insta_clone' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    },
    {
      _id: 'insta_clone.feedViewer',
      user: 'feedViewer',
      db: 'insta_clone',
      roles: [ { role: 'read', db: 'insta_clone' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    }
  ],
  ok: 1
}
```

Roles assigned to each user

User	Role	What the user can do
storyAdmin	dbAdmin (insta_clone)	Manage the database: create and drop indexes, view stats, manage collections. Cannot read or write documents.
feedViewer	read (insta_clone)	View documents (find) in the database. Cannot insert, update or delete.

Difference between 'read' and 'dbAdmin'

	read	dbAdmin
Purpose	Viewing data	Administering the database
Can read documents	Yes	No
Can modify documents	No	No
Can create/drop indexes, see stats	No	Yes
Typical user	Feed viewer, analyst, report tool	Database administrator

In my own words: The 'read' role is about the *data*: it lets a user look at the documents but nothing more. The 'dbAdmin' role is about the *database itself*: it lets a user handle administrative tasks such as indexes and statistics, but it does not give access to the data inside the collections. Each role gives only the permissions a person needs, which follows the principle of least privilege.